Вернуться к 'Рекурсия. Функции высших порядков'

Тест начат	Пятница, 25 Сентябрь 2026, 23:31
Состояние	Завершенные
Завершен	Суббота, 26 Сентябрь 2026, 00:14
Прошло времени	43 мин. 14 сек.
Баллы	15,00/15,00
Оценка	**5,00** из 5,00 (**100**%)

Вопрос **1**

Верно

Баллов: 1,00 из 1,00

Выражение `takeWhile even [2, 4, 5, 6, 7, 8]` равно

Выберите один ответ:

- `[2,4,6,8]`
- `[2]`
- `[]`
- `[2,4,6]`
- `[2,4]` ✔

Вопрос **2**

Верно

Баллов: 1,00 из 1,00

Выражение `filter even (map (+1) [1..5])` равно

Выберите один ответ:

- `[]`
- `[2,4]`
- `[2,4,6]` ✔
- `[3,5]`
- `[1,3,5]`

Вопрос **3**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию all

```
all :: (a -> Bool) -> [a] -> Bool
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один или несколько ответов:

- [ ] `all p = map p . and`
- [x] `all p = not . any (not . p)` ✔
- [ ] `all p xs = map p (and xs)`
- [x] `all p = and . map p` ✔
- [x] `all p = foldr (&&) True . map p` ✔
- [x] `all p xs = and (map p xs)` ✔
- [x] `all p xs = foldl (&&) True (map p xs)` ✔
- [ ] `all p xs = foldr (&&) False (map p xs)`

Вопрос **4**

Верно

Баллов: 1,00 из 1,00

Используя слдующее опрделеление, показать, как вычисляется

`drop 3 [1, 2, 3, 4, 5]`

```
drop :: Int -> [a] -> [a]
drop 0 xs = xs
drop n [] = []
drop n (_ : xs) = drop (n - 1) xs
```

Выберите один ответ:

a.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [2,3,4,5]
= { applying drop }
drop 1 [3,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[4,5]
```

✔

b.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [2,3,4,5]
= { applying drop }
drop 1 [3,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[5]
```

c.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [1,2,4,5]
= { applying drop }
drop 1 [1,4,5]
= { applying drop }
drop 0 [4,5]
= { applying drop }
[4,5]
```

d.

```
drop 3 [1,2,3,4,5]
= { applying drop }
drop 2 [1,2,3,4]
= { applying drop }
drop 1 [1,2,3]
= { applying drop }
drop 0 [1,2]
= { applying drop }
[1,2]
```

Вопрос **5**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая объединяет список списков:

```
concat :: [[a]] -> [a]
```

Выберите один ответ:

```
concat [] = []
concat (xs : xss) = xs ++ concat xss
```
✔

```
concat [[]] = []
concat (xs : xss) = xs ++ concat xss
```

```
concat [] = [[]]
concat (xs : xss) = xs ++ concat xss
```

```
concat [] = []
concat (xs : xss) = xs : concat xss
```

Вопрос **6**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции слияния

```
merge :: Ord a => [a] -> [a] -> [a]
```

которая объединяет два списка, отсортированных в порядке возрастания в результирующий отсортированный в порядке возрастания список. Например:

```
> merge [2, 5, 6] [1, 3, 4]
[1, 2, 3, 4, 5, 6]
```

Выберите один ответ:

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then y : merge xs (y : ys) else x : merge (x : x
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then x : merge xs (y : ys) else y : merge (x : x
```
✔

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then x : merge xs ys else y : merge xs ys
```

```
merge [] ys = ys
merge xs [] = xs
merge (x : xs) (y : ys)
  = if x <= y then y : merge (x : xs) ys else x : merge xs (y
```

Вопрос **7**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию filter

```
filter :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

-
```
filter p = foldl (\ xs x -> if p x then x : xs else xs) []
```
-
```
filter p = foldl (\ x xs -> if p x then xs ++ [x] else xs) []
```
-
```
filter p = foldr (\ x xs -> if p x then x : xs else xs) []
```
✔
-
```
filter p = foldr (\ x xs -> if p x then xs ++ [x] else xs) []
```

Вопрос **8**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию map

```
map :: (a -> b) -> [a] -> [b]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

-
```
map f = foldl (\ xs x -> xs ++ [f x]) []
```
✔
-
```
map f = foldr (\ x xs -> xs ++ [f x]) []
```
-
```
map f = foldl (\ xs x -> f x : xs) []
```
-
```
map f = foldr (\ x xs -> f x ++ xs) []
```

Вопрос **9**

Верно

Баллов: 1,00 из 1,00

Выражение `foldr (-) 0 [1, 2, 3, 4]` равно

Выберите один ответ:

- `-8`
- `-10`
- `10`
- `0`
- `-2` ✔

Вопрос **10**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая выбирает n-й элемент списка. Отсчет с 0.

```
(!!) :: [a] -> Int -> a
```

Выберите один ответ:

a.
```
(x : _) !! 0 = x
(_ : xs) !! n = xs !! (n - 1)
```
✔

b.
```
(x : _) !! 1 = x
(_ : xs) !! n = xs !! (n - 1)
```

c.
```
(x : _) !! 0 = x
(_ : xs) !! n = xs !! (n - 1)
[] !! n = 0
```

d.
```
(x : _) !! 0 = [x]
(_ : xs) !! n = xs !! (n - 1)
```

Вопрос **11**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию takeWhile

```
takeWhile :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

```
takeWhile _ [] = []
takeWhile p (x : xs)
  | p x = takeWhile p xs
  | otherwise = []
```

```
takeWhile p = foldl (\ acc x -> if p x then x : acc else acc)
```

```
takeWhile _ [] = []
takeWhile p (x : xs)
  | p x = x : takeWhile p xs
  | otherwise = []
```
✔

```
takeWhile _ [] = []
takeWhile p (x : xs)
  | p x = x : takeWhile p xs
  | otherwise = takeWhile p xs
```

Вопрос **12**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, покажите как вычисляется `init [1, 2, 3]`

```
init :: [a] -> [a]
init [_] = []
init (x : xs) = x : init xs
```

Выберите один ответ:

a.
```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : []
= { list notation }
[1,2]
```
✔

b.
```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : []
= { list notation }
[1]
```

c.
```
init [1,2,3]
= { applying init }
[1,2] : init [3]
= { applying init }
[1,2] : []
= { list notation }
[1,2]
```

d.
```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : 3 : init []
= { applying init }
1 : 2 : 3 : []
= { list notation }
[1,2,3]
```

Вопрос **13**

Верно

Баллов: 1,00 из 1,00

Выражение `zip [1, 2] ['a', 'b', 'c']` равно

Выберите один ответ:

- `An error`
- `([1,2],['a','b','c'])`
- `[(1,'a'),(2,'b'),(2,'c')]`
- `[(1,'a'),(2,'b'),(3,'c')]`
- `[(1,'a'),(2,'b')]` ✔

Вопрос **14**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, показать, как вычисляется

`length [1, 2, 3]`

```
length :: [a] -> Int
length [] = 0
length (_ : xs) = 1 + length xs
```

Выберите один ответ:

a.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + (1 + length []))
= { applying length }
1 + (1 + (1 + 0))
= { applying + }
1 + (1 + 1)
= { applying + }
1 + 2
= { applying + }
3
```

✔

b.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + 1)
= { applying + }
3
```

c.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + (1 + length []))
= { applying length }
1 + (1 + (1 + 1))
= { applying + }
1 + (1 + 2)
= { applying + }
1 + 3
= { applying + }
4
```

d.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (2 + length [3])
= { applying length }
1 + (2 + (3 + length []))
= { applying length }
1 + (2 + (3 + 0))
= { applying + }
6
```

Вопрос **15**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая определяет, является ли указанное значение элементом списка:

```
elem :: Eq a => a -> [a] -> Bool
```

Выберите один ответ:

```
elem _ [] = False
elem x (y : ys)
  | x == y = True
  | otherwise = elem x (y : ys)
```

```
elem _ [] = False
elem x (y : ys)
  | x == y = True
  | otherwise = elem x ys
```

✔

```
elem _ [] = False
elem x (y : ys) = x == y
```

```
elem _ [] = True
elem x (y : ys)
  | x == y = True
  | otherwise = elem x ys
```